

머신러닝 모델 설명

08/18/2026 16:52:42

<https://chatgpt.com/c/6a7e7068-e58c-83ee-8fb5-a4bc0f3254f7>

가능해. 이번에는 “코드 문법 설명”만 하지 않고, 각 챕터마다 왜 하는지 → 무엇인지 →...



가능해. 이번에는 “코드 문법 설명”만 하지 않고, 각 챕터마다 왜 하는지 → 무엇인지 → 코드 한 줄씩 → 결과를 어떻게 해석하는지 순서로 볼게. 특히 처음 나오는 모델(model), 학습(train), 예측(predict), 평가(metric) 같은 용어도 먼저 아주 쉽게 잡아줄게.

파일을 확인해보니 수업 흐름이 꽤 명확해. ① scikit-learn으로 첫 모델 만들기 → ② K-Fold로 더 믿을 만하게 평가하기 → ③ 문자열 데이터를 숫자로 바꾸기 → ④ Titanic 데이터에 전처리+여러 모델 적용하기 순서야.

그래서 이 순서를 그대로 따라가면서, 먼저 “모델이 대체 뭐가?”부터 연결해서 설명하겠다.

Thinking

- Interacted with Files



압축파일 내용 목록 확인

- 압축파일 목록 확인 및 노트북 코드와 마크다운 분석



노트북 출력 확인 및 챕터별 코드 설명

- 정리하며 설명하기

머신러닝의 흐름은 `데이터 준비 → 학습·평가 분리 → 모델 학습 → 예측 → 정확도 계산`입니다. 이어서 각 파일을 이 흐름에 맞춰, 변수와 반복문의 역할까지 차례대로 설명하겠습니다.

- 정리하며 설명했다

- 확인하고 타이타닉 전처리 및 예측 결과를 검증함

✓ 1m 15s 동안 처리함

응. 파일 안의 노트북 4개를 전부 확인했어. 이 DAY1 수업은 사실 아래 하나의 머신러닝 흐름을 단계별로 나눈 거야.

데이터 준비 → 학습/평가 데이터 분리 → 모델 선택 → 모델 학습 → 예측 → 성능 평가

먼저 이 흐름을 이해하면 코드가 훨씬 덜 외계어처럼 보여.

0. 먼저 "모델"이 뭐가?

머신러닝에서 **모델(Model)**은 아주 쉽게 말하면



데이터를 보고 규칙을 학습한 뒤, 새로운 데이터의 답을 예측하는 프로그램

이라고 생각하면 돼.

예를 들어 붓꽃 데이터가 있다고 하자.

꽃받침 길이	꽃받침 너비	꽃잎 길이	꽃잎 너비	꽃 종류
5.1	3.5	1.4	0.2	setosa
6.5	3.0	5.2	2.0	virginica

우리가 모델에게 수많은 예시를 보여준다.

```
[5.1, 3.5, 1.4, 0.2] → setosa
[6.5, 3.0, 5.2, 2.0] → virginica
...
```

그러면 모델이 이런 관계를 스스로 찾아낸다.

그리고 처음 보는 꽃:

```
[5.0, 3.4, 1.5, 0.2] → ???
```

를 넣었을 때

```
setosa
```

라고 예측하는 거야.

그래서 앞으로 아주 자주 보는 세 단어가 있다.

```
model.fit(X_train, y_train)
model.predict(X_test)
accuracy_score(y_test, pred)
```

뜻은 딱 이것이다.

```
fit      → 공부시킨다
predict  → 문제를 풀게 한다
accuracy → 채점한다
```

이 세 개는 꼭 잡고 가자.

Chapter 1. 01_sklearn.ipynb

처음으로 머신러닝 모델 만들어보기

왜 하는가?

이 챕터의 목적은 머신러닝의 전체 과정을 한 번 처음부터 끝까지 경험하는 것이다.

```
데이터 가져오기
↓
문제(x) / 정답(y) 만들기
↓
학습용 / 시험용 나누기
↓
모델 만들기
↓
학습
↓
예측
↓
정확도 확인
```

DAY1에서 가장 중요한 챕터라고 봐도 돼.

1-1. 라이브러리 가져오기

```
import sklearn
```

`scikit-learn` 이라는 머신러닝 라이브러리를 가져온다.

파이썬이 기본적으로 `DecisionTree`나 `train_test_split`을 알고 있는 것은 아니기 때문에 라이브러리가 필요하다.

```
from sklearn.datasets import load_iris
```

`scikit-learn` 안에 들어있는 **붓꽃 예제 데이터**를 가져오기 위한 함수다.

```
load_iris()
```

하면 붓꽃 데이터가 나온다.

```
from sklearn.tree import DecisionTreeClassifier
```

****Decision Tree(결정트리)****라는 머신러닝 모델을 가져온다.

Decision Tree는 쉽게 말하면 계속 질문을 던진다.

꽃잎 길이가 2.45 이하인가?

↓

YES / NO

그리고 조건을 계속 나눠서 최종적으로 꽃의 종류를 결정한다.

즉,

DecisionTreeClassifier = 분류 문제를 풀기 위한 결정트리 모델

이다.

여기서 `Classifier` 가 붙은 이유는 **분류** 문제이기 때문이야.

붓꽃 종류:

```
setosa  
versicolor  
virginica
```

중 하나를 맞히는 문제니까 ****분류(Classification)****다.

```
from sklearn.model_selection import train_test_split
```

데이터를

```
공부용 데이터  
시험용 데이터
```

로 나누기 위한 함수다.

왜 나눠야 할까?

모델이 공부한 문제를 그대로 시험 보면 안 되기 때문이다.

사람으로 생각하면

```
문제 100개를 정답까지 외움  
→ 똑같은 100개 시험  
→ 100점
```

이라고 해서 실력이 좋다고 말하기 어렵지.

그래서 모델이 **처음 보는 데이터**를 얼마나 잘 맞히는지 확인한다.

```
import pandas as pd
```

표 형태 데이터를 다루는 Pandas를 가져온다.

`pd` 는 pandas의 별명이다.

1-2. Iris 데이터 확인

```
print(load_iris())
```

붓꽃 데이터 전체 구조를 출력한다.

안에는 대표적으로

```
data  
target  
target_names  
feature_names
```

같은 정보가 들어 있다.

1-3. X와 y 만들기

```
iris = load_iris()
```

붓꽃 데이터를 가져와서 `iris` 변수에 저장한다.

```
iris_data = iris.data
```

여기에는 꽃의 특징이 들어간다.

예를 들면:

```
꽃받침 길이  
꽃받침 너비  
꽃잎 길이  
꽃잎 너비
```

머신러닝에서는 이런 **예측에 사용하는 정보**를 보통

Feature(피쳐, 특성)

라고 부른다.

그리고 흔히 `x` 라고 표현한다.

```
iris_label = iris.target
```

꽃의 실제 종류, 즉 **정답**이다.

```
0 = setosa  
1 = versicolor  
2 = virginica
```

정답은 흔히 `y` 라고 표현한다.

그래서 머신러닝 코드에서 엄청 자주 보는 것이

```
x  
y
```

다.

기억

```
x = 문제  
y = 정답
```

이라고 우선 생각하면 된다.

1-4. 학습용 / 시험용 데이터 나누기

```
X_train, X_test, y_train, y_test = train_test_split(  
    iris_data,  
    iris_label,  
    test_size=0.2,  
    random_state=42  
)
```

여기가 처음 보면 어렵다.

하나씩 보면,

```
iris_data
```

문제 데이터 X.

```
iris_label
```

정답 데이터 y.

```
test_size=0.2
```

전체 데이터 중 **20%**를 **시험용**으로 쓰겠다는 뜻.

따라서

```
80% → 학습  
20% → 테스트
```

가 된다.

붓꽃 데이터는 150개니까 대략:


```
X_train = 120개 문제
y_train = 120개 정답

X_test = 30개 문제
y_test = 30개 정답
```

```
random_state=42
```

데이터를 랜덤하게 나눌 때 **매번 같은 방식으로 나누도록 고정한다.**

42라는 숫자 자체에 특별한 의미는 없다.

```
random_state=1
random_state=10
random_state=42
```

어떤 숫자든 가능하다.

중요한 건 **결과 재현**이다.

1-5. 모델 만들기

```
dt_clf = DecisionTreeClassifier(random_state=42)
```

결정트리 모델을 하나 만든다.

아직 공부는 안 했다.

이 시점은

머리는 만들었지만 아직 아무것도 배우지 않은 상태

라고 보면 된다.

`dt_clf` 는 변수 이름일 뿐이다.

```
dt = Decision Tree
clf = Classifier
```

정도로 지은 이름이다.

1-6. 모델 학습

```
dt_clf.fit(X_train, y_train)
```

여기가 머신러닝에서 매우 중요하다.

```
fit()
```

은

모델에게 데이터를 학습시킨다

는 뜻이다.

모델에게

```
X_train = 문제  
y_train = 정답
```

을 같이 보여준다.

즉,

```
이 특징을 가진 꽃의 정답은 0이야.  
이 특징을 가진 꽃의 정답은 2야.  
...
```

를 공부하는 과정이다.

핵심

```
모델.fit(문제, 정답)
```

이라고 기억하면 된다.

1-7. 예측

```
pred = dt_clf.predict(X_test)
```

이번에는 정답을 알려주지 않는다.

모델에게

```
X_test
```

만 주고

"너 혼자 맞춰봐."

하는 것이다.

그 결과를

```
pred
```

에 저장한다.

```
print(pred)
```

모델이 예측한 답을 출력한다.

실제 파일에서는

```
[1 0 2 1 1 0 ...]
```

처럼 나온다.

1-8. 모델 채점

```
from sklearn.metrics import accuracy_score
```

정확도를 계산하는 함수를 가져온다.

```
print(accuracy_score(y_test, pred))
```

비교하는 것은:

```
y_test = 실제 정답
pred   = 모델이 예측한 답
```

이다.

즉,

```
실제 답 : 0 1 2 1 0
모델 답  : 0 1 2 0 0
```

같이 비교해서 얼마나 맞았는지 계산한다.

파일에서는:

```
1.0
```

이 나왔는데,

정확도 100%

라는 뜻이다.

다만 Iris 데이터 자체가 비교적 쉬운 데이터라 가능한 결과이지, 일반적으로 모델 정확도가 항상 100%인 것은 아니다.

Chapter 1 핵심

이 4줄은 앞으로 계속 나온다.

```
X_train, X_test, y_train, y_test = train_test_split(X, y)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

pred = model.predict(X_test)

accuracy_score(y_test, pred)
```

머릿속으로 번역하면:

문제와 정답을 공부용/시험용으로 나눈다.

모델을 만든다.

공부시킨다.

시험 문제를 풀게 한다.

채점한다.

Chapter 2. `02_kfold.ipynb`

K-Fold 교차검증

왜 하는가?

Chapter 1에서는 한 번만 데이터를 나눴다.

그런데 운이 좋게 쉬운 문제만 시험 데이터에 들어갈 수도 있다.

그래서

"시험을 한 번만 보지 말고 여러 번 봐보자."

라는 것이 **교차검증(Cross Validation)**이다.

K-Fold란?

예를 들어 데이터를 5등분한다.

① ② ③ ④ ⑤

첫 번째 시험:

시험: ①

학습: ②③④⑤

두 번째:

시험: ②

학습: ①③④⑤

계속 반복해서 5번 시험을 본다.

마지막에

5번의 정확도 평균

을 구한다.

코드

```
from sklearn.model_selection import KFold
```

K-Fold 기능을 가져온다.

```
kfold = KFold(n_splits=5)
```

데이터를 **5조각으로** 나누겠다는 뜻.

```
cv_accuracy = []
```

5번 시험한 정확도를 저장할 빈 리스트.

예를 들면 나중에

```
[1.0, 1.0, 0.8333, 0.9333, 0.8]
```

이렇게 된다.

```
print(features.shape[0])
```

`shape` 는 데이터의 크기를 본다.

Iris가

```
(150, 4)
```

라면

```
shape[0]
```

은 행 개수인

```
150
```

이다.

가장 중요한 반복문

```
for train_index, test_index in kfold.split(features):
```

KFold가

```
이번에 학습할 행 번호  
이번에 시험 볼 행 번호
```

를 만들어준다.

```
X_train, X_test = features[train_index], features[test_index]
```

그 번호를 이용해 실제 X 데이터를 꺼낸다.

```
y_train, y_test = label[train_index], label[test_index]
```

마찬가지로 정답도 나눈다.

```
dt_clf.fit(X_train, y_train)
```

학습.

```
pred = dt_clf.predict(X_test)
```

예측.

Chapter 1과 똑같지?

KFold는 결국

학습 → 예측 → 채점

을 여러 번 하는 것이다.

```
accuracy = np.round(
    accuracy_score(y_test, pred),
    4
)
```

정확도를 구하고 소수점 4자리까지 반올림한다.

```
cv_accuracy.append(accuracy)
```

정확도를 리스트에 하나씩 넣는다.

마지막:

```
print(np.mean(cv_accuracy))
```

5개의 정확도 평균을 계산한다.

파일에서는 약

```
0.91332
```

즉 약 ****91.3%****다.

핵심

한 번 평가 → 결과가 우연일 수 있음

여러 번 나눠 평가 → 조금 더 신뢰할 만함

이게 교차검증을 하는 이유다.

참고로 분류에서는 이후에 **StratifiedKFold**도 배우게 될 가능성이 높아. 클래스 비율까지 비슷하게 나눠주는 방식이다.

Chapter 3. `03_data.ipynb`

문자열을 숫자로 바꾸기 왜 하는가?

머신러닝 모델은 보통 이런 데이터는 잘 처리한다.

```
나이 = 30
온도 = 75.2
가격 = 12000
```

그런데

```
성별 = 남자
제품 = 냉장고
지역 = 천안
```

처럼 문자로 된 데이터는 그대로 모델에 넣기 어려운 경우가 많다.

그래서 숫자로 변환한다.

이것을

Encoding(인코딩)

이라고 한다.

3-1. LabelEncoder

```
from sklearn.preprocessing import LabelEncoder
```

문자 범주를 숫자로 바꾸는 도구다.

```
items = ["TV", "냉장고", "컴퓨터", "선풍기", ...]
```

문자 데이터.

```
encoder = LabelEncoder()
```

인코더를 만든다.

```
encoder.fit(items)
```

어떤 종류가 있는지 학습한다.

예:

```
TV  
냉장고  
선풍기  
에어컨  
컴퓨터
```

```
labels = encoder.transform(items)
```

실제로 숫자로 변환한다.

결과:

```
[0 1 4 2 2 3 3]
```

여기서 숫자가 크다고 더 좋은 제품이라는 뜻은 **절대 아니다**.

그냥 번호를 붙인 것이다.

3-2. One-Hot Encoding

LabelEncoding에는 문제가 하나 있다.

```
TV = 0  
냉장고 = 1
```

선풍기 = 2
에어컨 = 3

라고 하면 모델이 경우에 따라

에어컨 3 > TV 0

처럼 숫자에 순서가 있다고 오해할 수 있다.

그래서 사용하는 대표적인 방법이 **One-Hot Encoding**이다.

예:

	TV	냉장고	선풍기
TV	1	0	0
냉장고	0	1	0
선풍기	0	0	1

```
items = np.array(items).reshape(-1, 1)
```

여기가 어려워 보인다.

원래:

```
["TV", "냉장고", "컴퓨터"]
```

1차원 데이터를

```
[  
  ["TV"],  
  ["냉장고"],  
  ["컴퓨터"]  
]
```

처럼 **세로 형태의 2차원 데이터**로 바꾼다.

OneHotEncoder가 2차원 형태를 기대하기 때문이다.

```
oh_encoder = OneHotEncoder()
```

원핫 인코더를 만든다.

```
oh_encoder.fit(items)
```

어떤 카테고리가 있는지 확인.

```
oh_labels = oh_encoder.transform(items)
```

실제로 원핫 인코딩.

```
print(oh_labels.toarray())
```

일반 배열 형태로 출력한다.

```
print(oh_labels.shape)
```

결과가

```
(7, 5)
```

이다.

왜냐하면

```
데이터 7개  
제품 종류 5개
```

이기 때문이다.

3-3. Pandas로 더 쉽게 하기

```
df = pd.DataFrame({  
    "item": ["TV", "냉장고", ...]  
})
```

DataFrame을 만든다.

```
pd.get_dummies(df)
```

Pandas가 자동으로 One-Hot Encoding을 해준다.

그래서 실제 데이터 분석에서는 이것도 상당히 자주 본다.

Chapter 4. 04_titanic.ipynb

실제 데이터로 머신러닝 해보기

여기부터 Chapter 1~3이 합쳐진다.

목표

승객 정보를 보고

이 승객이 생존했는가?

를 예측하는 모델을 만드는 것이다.

즉 정답은

```
Survived  
0 = 사망  
1 = 생존
```

두 개이므로 **이진 분류** 문제다.

4-1. 데이터 읽기

```
titanic_df = pd.read_csv("./data/train.csv")
```

CSV 파일을 Pandas로 읽는다.

```
print(titanic_df.head())
```

앞의 5행을 확인한다.

여기서 EDA할 때 배웠던 Pandas가 바로 등장한다.

4-2. X와 y 나누기

```
y_titanic_df = titanic_df["Survived"]
```

우리가 맞혀야 하는 정답.

즉:

```
y = 생존 여부
```

```
X_titanic_df = titanic_df.drop("Survived", axis=1)
```

`Survived` 를 제외한 나머지를 문제 X로 사용한다.

즉:

```
Pclass
Sex
Age
Fare
Cabin
...
↓
생존 여부 예측
```

이다.

이것도 Chapter 1의

```
iris_data = X
iris_target = y
```

와 완전히 같은 구조다.

4-3. 결측치 확인

```
titanic_df.info()
```

컬럼 타입과 결측치를 확인한다.

실제로:

```
Age      → 결측치 있음  
Cabin    → 결측치 매우 많음  
Embarked → 약간 있음
```

을 확인할 수 있다.

4-4. Kaggle test 데이터

```
test_df = pd.read_csv("./data/test.csv")
```

여기서 조금 헷갈릴 수 있다.

Titanic에는 원래

```
train.csv  
test.csv
```

가 있다.

`train.csv`에는 `Survived` 정답이 있고,

`test.csv`에는 정답이 없다.

그래서

```
train.csv → 모델 만들기  
test.csv  → 최종 생존 여부 예측
```

용도다.

```
res = test_df["PassengerId"]
```

승객 번호를 따로 보관한다.

왜냐하면 뒤에서 일부 컬럼을 삭제하더라도 최종 결과에는 `PassengerId`가 필요하기 때문이다.

4-5. 결측치 처리

```
X_titanic_df["Age"] = X_titanic_df["Age"].fillna(  
    X_titanic_df["Age"].mean()  
)
```

Age가 비어 있으면 평균 나이로 채운다.

예:

```
22  
35  
NaN  
40
```

이라면 NaN을 평균값으로 대체한다.

```
X_titanic_df["Cabin"] = X_titanic_df["Cabin"].fillna("N")
```

객실 정보가 없으면 "N".

```
X_titanic_df["Embarked"] = X_titanic_df["Embarked"].fillna("N")
```

탑승항 정보가 없으면 "N".

```
X_titanic_df["Fare"] = X_titanic_df["Fare"].fillna(0)
```

요금 정보가 없으면 0으로 채운다.

Cabin에서 첫 글자만 남기기

```
X_titanic_df["Cabin"] = X_titanic_df["Cabin"].str[:1]
```

예:

```
C85 → C  
C123 → C  
E46 → E
```


왜?

객실 번호 하나하나보다

어느 **Deck**에 있었느냐

정도만 사용하는 것이다.

이것도 넓게 보면 **Feature Engineering**이다.

즉 원래 데이터를 모델이 사용하기 좋은 정보로 가공하는 것이다.

4-6. 필요 없는 컬럼 제거

```
drop_col = ["PassengerId", "Name", "Ticket"]
```

```
X_titanic_df = X_titanic_df.drop(columns=drop_col)
```

PassengerId, Name, Ticket을 제거한다.

초기 실습에서는 고유값이 많고 그대로 사용하기 어려운 컬럼을 빼는 것이다.

4-7. 문자 → 숫자

```
oe = OrdinalEncoder(  
    handle_unknown="use_encoded_value",  
    unknown_value=-1  
)
```

OrdinalEncoder를 만든다.

Titanic에는

```
Sex  
Cabin  
Embarked
```

처럼 문자가 있다.

그래서 숫자로 변환한다.

```
oe.fit_transform(  
    X_titanic_df[["Cabin", "Sex", "Embarked"]]  
)
```

`fit_transform()` 은

```
fit()  
+  
transform()
```

을 한 번에 하는 것이다.

즉:

```
어떤 값이 존재하는지 학습  
+  
숫자로 변환
```

이다.

반면 test 데이터에는:

```
oe.transform(test_df[["Cabin", "Sex", "Embarked"]])
```

만 사용한다.

이 차이가 매우 중요하다.

```
train 데이터 → fit + transform  
test 데이터 → transform만
```

왜냐하면 전처리 규칙도 **train** 데이터에서 학습해야 하기 때문이다.

4-8. 다시 Train / Validation 분리

```
X_train, X_vaild, y_train, y_valid = train_test_split(  
    X_titanic_df,  
    y_titanic_df,  
    test_size=0.2,
```

```
random_state=42
)
```

여기서 또 test처럼 보이는 개념이 나와서 헷갈릴 수 있다.

현재 구조는 사실:

```
원래 train.csv
    ↓
X_train 80%
X_valid 20%

원래 test.csv
    ↓
최종 예측용
```

이다.

`valid` 는 **Validation**, 검증 데이터라는 뜻이다.

그리고 코드의

```
X_vaild
```

는 `valid` 의 철자가 살짝 잘못된 변수명이다.

코드가 계속 같은 이름을 사용해서 실행 자체에는 문제없다.

4-9. 세 가지 모델 비교

여기가 이번 파일에서 가장 중요한 부분 중 하나다.

```
dt_clf = DecisionTreeClassifier(random_state=42)
```

Decision Tree

```
조건 질문
↓
조건 질문
↓
조건 질문
```

↓
생존/사망

형태.

```
rf_clf = RandomForestClassifier(random_state=42)
```

Random Forest

결정트리를 여러 개 만든 다음 투표한다.

예:

Tree 1 → 생존
Tree 2 → 생존
Tree 3 → 사망
Tree 4 → 생존

최종 → 생존

그래서 이름도

Decision Tree = 나무 한 그루
Random Forest = 나무 여러 그루 → 숲

이다.

이게 수업에서 배운 **Bagging** 계열과 연결된다.

```
lr_clf = LogisticRegression(solver="liblinear")
```

Logistic Regression

이름에 Regression이 들어 있지만 주로 **분류 모델**이다.

여러 특징을 이용해서

생존할 가능성

을 계산하고,

그 확률을 바탕으로

```
0 / 1
```

을 결정한다.

4-10. Decision Tree 학습/예측

```
dt_clf.fit(X_train, y_train)
```

공부.

```
df_pred = dt_clf.predict(X_vaild)
```

시험.

```
accuracy_score(y_valid, df_pred)
```

채점.

정확도는 약:

```
0.771  
→ 77.1%
```

Random Forest

```
rf_clf.fit(X_train, y_train)  
rf_pred = rf_clf.predict(X_vaild)  
print(accuracy_score(y_valid, rf_pred))
```

약:

```
80.4%
```

Logistic Regression

```
lr_clf.fit(X_train, y_train)
lr_pred = lr_clf.predict(X_vaild)
print(accuracy_score(y_valid, lr_pred))
```

약:

79.3%

따라서 이 Validation에서는:

Decision Tree	77.1%
Logistic Regression	79.3%
Random Forest	80.4%

Random Forest가 가장 높게 나온다.

다만 **한 번의 결과만으로 Random Forest가 무조건 최고라고 결론내리면 안 된다.**

그래서 앞 Chapter에서 K-Fold를 배운 것이다.

연결되지?

4-11. 진짜 test 데이터 예측

```
dt_test = dt_clf.predict(test_df)
rf_test = rf_clf.predict(test_df)
lr_test = lr_clf.predict(test_df)
```

이번에는 정답이 없는 Titanic `test.csv` 의 생존 여부를 예측한다.

```
result = pd.DataFrame({
    "PassengerId": res,
    "DecisionTree": dt_test,
    "RandomForest": rf_test,
    "LogisticRegression": lr_test
})
```

결과를 하나의 표로 만든다.

예:

PassengerId	DecisionTree	RandomForest	LogisticRegression
892	1	0	0
893	0	1	1

이런 식이다.

그런데 Titanic 노트북에서 하나 주의할 점

파일에 저장되어 있는 일부 출력 결과가 현재 코드와 맞지 않는다.

특히 뒤쪽에 RandomForest가 거의 전부 1, LogisticRegression이 전부 1이라고 저장되어 있는데, 내가 원본 CSV부터 코드를 순서대로 다시 실행해보면:

```
DecisionTree
0: 256명
1: 162명

RandomForest
0: 266명
1: 152명

LogisticRegression
0: 266명
1: 152명
```

으로 나온다.

즉 코드 자체보다 **Jupyter Notebook**을 중간에 수정하거나 셀 실행 순서를 바꾸면서 예전 출력이 남아 있는 것으로 보인다.

Jupyter에서 굉장히 흔한 현상이다.

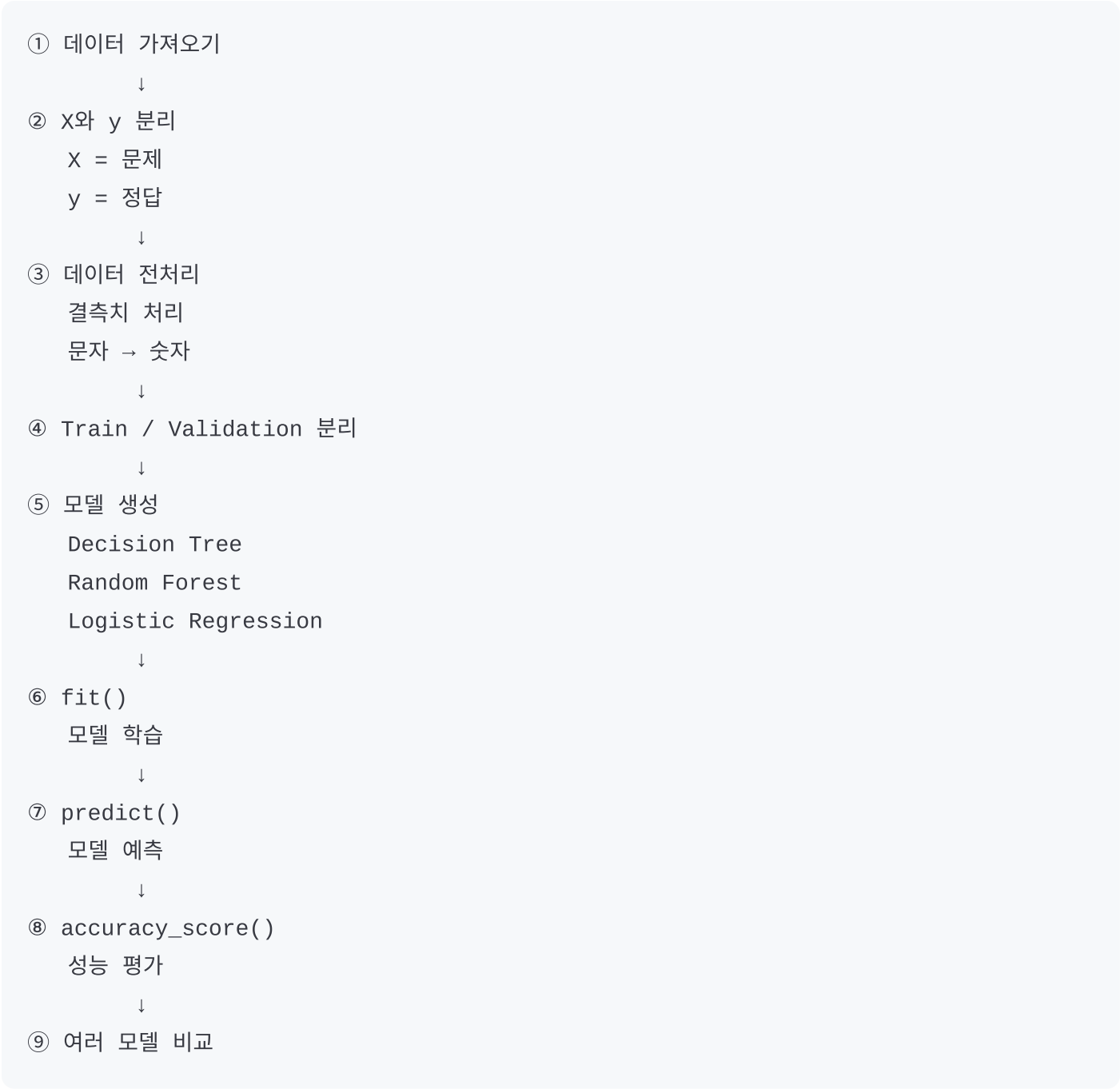
그래서 노트북 공부할 때는 필요하면:

```
Restart Kernel
→ Run All
```

로 위에서부터 다시 실행하는 습관이 좋다.

DAY1 전체를 한 장으로 보면

이걸 가장 중요하게 기억하면 된다.



그리고 머신러닝에서 지금 당장 가장 먼저 익숙해져야 할 단어는 이 정도야.

용어	지금은 이렇게 이해
Feature(X)	모델에게 주는 문제/정보
Label/Target(y)	모델이 맞혀야 하는 정답
Model	데이터에서 규칙을 학습해서 예측하는 프로그램
Train	모델 공부시키기

용어	지금은 이렇게 이해
fit()	학습시키기
predict()	새로운 문제 풀게 하기
Test/Validation	모델 시험보기
Accuracy	몇 % 맞혔는지
Preprocessing	모델에 넣기 좋게 데이터 정리
Encoding	문자 데이터를 숫자로 바꾸기
Decision Tree	조건을 나눠가며 판단
Random Forest	여러 결정트리가 투표
Logistic Regression	특징을 이용해 분류 확률 계산
K-Fold	데이터를 바꿔가며 여러 번 시험

지금 단계에서는 코드 자체를 외우는 것보다 `x → split → model → fit → predict → 평가` 흐름을 이해하는 게 훨씬 중요해. 이 DAY1 자료는 사실 그 흐름을 반복해서 익히게 만든 수업이라고 보면 된다.

2026. 8. 14. 오전 10:33:53